

WholeSync

A B2B Wholesale Marketplace for Retailers and Wholesalers

Project Documentation

Team Members	Role
Aswin I.	Developer
Karthikeyan S.	Developer
Naveen Kumar P.	Developer
Saravanan P.	Developer
Sivadurai R.	Developer
Dr. T. Mahendran	Project Guide

Certificate

This project is submitted in partial fulfillment of the requirements for the academic programme in Computer Science and Engineering.

Project Title: WholeSync

Student Team:

- Aswin I.
- Karthikeyan S.
- Naveen Kumar P.
- Saravanan P.
- Sivadurai R.

Project Guide: Dr. T. Mahendran

Date: _____

Signature of Guide: _____

Declaration

We, the undersigned students, declare that the project work titled “WholeSync” is our original work and has not been submitted elsewhere for academic credit.

Acknowledgement

We sincerely thank our project guide, Dr. T. Mahendran, for constant guidance, encouragement, and support throughout the project work.

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope and Features
6. Database Schema
7. Modules and Functional Description
8. System Design and Architecture
9. Implementation Details
10. Core Code Snippets
11. Screenshot Section
12. Testing and Validation
13. Deployment and Run Guide
14. Future Enhancements
15. Conclusion
16. References

Abstract

WholeSync is a web-based B2B marketplace platform designed to connect wholesalers and retailers across Tamil Nadu. The application enables wholesalers to manage inventory and maintain business profiles, while retailers can search, compare, bookmark, and contact sellers easily. The system is developed using Django and SQLite and is designed to be scalable, secure, and practical for local trade.

Introduction

The project addresses the common problem of fragmented trade communication between wholesalers and retailers. Many local businesses rely on informal communication and lack a digital marketplace that is easy to use. WholeSync provides a structured platform that improves product visibility, trust, and regional business efficiency.

Problem Statement

Retailers often face difficulty locating reliable sellers and comparing prices. Wholesalers also need a digital presence to publish inventory and reach more buyers. WholeSync fills this gap by providing a role-based B2B marketplace with inventory management, compare features, and seller location visibility.

Objectives

- To build a digital marketplace connecting wholesalers and retailers.
- To provide secure and role-based registration and login.
- To enable wholesaler stock management and business profile maintenance.
- To allow retailers to search, filter, compare, and bookmark products.
- To provide map-based shop location view for trusted seller discovery.
- To develop a functional and professional academic project application.

Scope and Features

The application is designed for Tamil Nadu-based wholesale and retail trade. It includes seller inventory management, retailer search, bookmark system, price comparison, map-based location visibility, and a responsive web interface. The system can also be extended to include payment integration, messaging, and analytics in the future.

Core Features

- Role-based registration for sellers and retailers
- Email verification for account authenticity
- Seller dashboard for managing stock
- Bulk stock upload support
- Retailer search with district and category filters
- Bookmarking and comparison functionality
- Map-based seller location selection and viewing
- Responsive design for desktop and mobile use

Database Schema

The database is implemented using Django ORM, with SQLite used in the development environment. The schema is designed to support user roles, seller profiles, stock items, bookmarks, and email verification tokens.

Table Name	Purpose	Key Fields
CustomUser	Stores user login and role information	id, username, email, role, email_verified
EmailVerificationToken	Stores email verification tokens	id, user_id, token, created_at
SellerProfile	Stores seller profile data and location	id, user_id, business_name, address, district, latitude, longitude
StockItem	Stores seller product records	id, seller_id, name, category, price, quantity, description
Bookmark	Stores retailer favorites	id, user_id, item_id, created_at

Relationship Summary:

- CustomUser to SellerProfile: One-to-One
- CustomUser to EmailVerificationToken: One-to-Many
- CustomUser to StockItem: One-to-Many
- CustomUser to Bookmark: One-to-Many
- StockItem to Bookmark: One-to-Many

Database Model Code

```
# marketplace/models.py
class CustomUser(AbstractUser):
    ROLE_CHOICES = [
        ('seller', 'Wholesale Seller'),
        ('retailer', 'Retailer'),
    ]
    role = models.CharField(max_length=10, choices=ROLE_CHOICES)
    email = models.EmailField(unique=True)
    email_verified = models.BooleanField(default=False)

class SellerProfile(models.Model):
    user = models.OneToOneField(CustomUser, on_delete=models.CASCADE, related_name='seller_profile')
    business_name = models.CharField(max_length=200)
    address = models.TextField()
    district = models.CharField(max_length=50)
    latitude = models.DecimalField(max_digits=9, decimal_places=6, null=True, blank=True)
    longitude = models.DecimalField(max_digits=9, decimal_places=6, null=True, blank=True)

class StockItem(models.Model):
    seller = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='stock_items')
    name = models.CharField(max_length=200)
    category = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    quantity = models.PositiveIntegerField()
    description = models.TextField(blank=True)
```

Modules and Functional Description

Authentication Module

Handles registration, login, logout, and email verification.

Seller Management Module

Allows wholesalers to add, update, and delete stock entries.

Retailer Search Module

Provides product search, district filtering, and comparison features.

Bookmark Module

Stores retailers' favourite product lists.

Map Location Module

Lets sellers pin their location and lets retailers view the location on a map.

Admin Module

Provides backend data management through Django admin.

System Design and Architecture

The application follows a standard Django three-tier model: frontend, application logic, and database layer. HTML pages are rendered using Django templates, requests are handled in view files, and all persistent data is stored in SQLite tables generated from models.

Technology Stack

- Frontend: HTML, CSS, JavaScript
- Backend: Django 5.2, Python 3.11+
- Database: SQLite for development
- Mapping: Leaflet.js and OpenStreetMap
- Security: Role validation, email verification, rate limiting

Implementation Details

The project was implemented in logical phases: authentication, seller dashboard, retailer features, and map integration. Each phase was tested and refined to improve usability and professionalism.

Main Code Snippets

```
# marketplace/urls.py
urlpatterns = [
    path('', home, name='home'),
    path('login/', user_login, name='login'),
    path('register/', user_register, name='register'),
    path('search/', search, name='search'),
    path('bookmarks/', bookmarks_view, name='bookmarks'),
    path('compare/', compare_view, name='compare'),
    path('seller/location/<int:user_id>/', seller_location, name='seller_location'),
    path('seller/profile/', seller_profile, name='seller_profile'),
]

# marketplace/settings.py
DATABASES = {
    'default': {
        'ENGINE': os.environ.get('DATABASE_ENGINE', 'django.db.backends.sqlite3'),
        'NAME': os.environ.get('DATABASE_NAME', str(BASE_DIR / 'db.sqlite3')),
    }
}
```

Project Screenshots

This section is reserved for the actual project screenshots to be inserted in the final printed copy of the documentation.

Screenshot 1: Project Interface

Home page / Registration page / Seller dashboard / Map view

Screenshot 2: Project Interface

Home page / Registration page / Seller dashboard / Map view

Screenshot 3: Project Interface

Home page / Registration page / Seller dashboard / Map view

Screenshot 4: Project Interface

Home page / Registration page / Seller dashboard / Map view

Testing and Validation

The application was validated by testing the major project flows and ensuring that core operations behave correctly. The checks include registration, login, stock management, search, comparison, bookmark usage, and seller location mapping.

- User registration and login work correctly.
- Sellers can add and edit stock items.
- Retailers can search and compare products.
- Bookmarks save and display properly.
- Seller location is stored and displayed on the map.
- Password visibility toggle works on login and registration forms.

Deployment and Run Guide

The project can be run locally through the Django development server. The commands below are used for setup and execution.

```
cd "d:\whole-sync\pocket-santhai"  
python -m venv .venv  
.\.venv\Scripts\Activate.ps1  
python -m pip install -r requirements.txt  
python manage.py migrate  
python manage.py runserver 0.0.0.0:8000
```

Future Enhancements

- Secure order placement and payment gateway integration.
- Real-time chat between sellers and retailers.
- Analytics dashboards for sales and stock trends.
- Multi-language support for Tamil and English.
- Expanded logistics and delivery integration.

Conclusion

WholeSync is a practical and meaningful B2B marketplace application that connects wholesalers and retailers while improving transparency, convenience, and efficiency in local trading. It demonstrates the use of Django framework, database design, authentication, and responsive web development in a real-world scenario. The project is suitable for academic evaluation, further enhancement, and practical deployment.

References

1. Django Documentation
2. SQLite Documentation
3. OpenStreetMap and Leaflet JS Documentation
4. Python Official Documentation

Submitted by:

Aswin I., Karthikeyan S., Naveen Kumar P., Saravanan P., Sivadurai R.

Guided by: Dr. T. Mahendran